

Accumulator-Based CPU and Single-Address Instructions

Key Concept: Accumulator-Based CPU & Single-Address Instructions

In typical computations such as $Z := X + Y$, three operands are involved:

- **Operand 1:** X
- **Operand 2:** Y
- **Destination:** Z

However, in **accumulator-based CPUs**, each instruction operates on:

- One operand from memory.
- The other operand is **implicitly the accumulator (AC)**.

Thus, a computation like $Z := X + Y$ must be broken into multiple instructions.

Step-by-Step Breakdown

HDL Format	Assembly	Description
AC := M(X)	LD X	Load value of X into the accumulator.
DR := AC	MOV DR, AC	Copy accumulator value into Data Register.
AC := M(Y)	LD Y	Load value of Y into the accumulator.
AC := AC + DR	ADD	Add DR to accumulator (i.e., $Y + X$).
M(Z) := AC	ST Z	Store result from accumulator to memory location Z.

Table 1: Instruction Breakdown for $Z := X + Y$ in Single-Address CPU

Instruction Consideration and Optimization

Instructions generally follow the form:

$$AC := f_i(AC, M(\text{adr}))$$

This means:

- Perform an operation f_i (e.g., ADD, SUB) between:
 - The value in the accumulator (AC)
 - A value from memory at address `adr`
- The result is stored back in the accumulator.

Implications

1. An extra register (e.g., DR) is often needed to hold intermediate results.
2. Instruction decoding becomes more complex due to implicit and explicit memory references.
3. Performance is affected by:
 - Increased memory accesses.
 - More instructions needed for a single logical task.

Summary: Why This Matters

- **Accumulator-based CPUs** simplify hardware by using fewer registers, but increase instruction count.
- **HDL (Hardware Description Language)** shows internal operations of CPU hardware.
- **Assembly Language** is what the programmer writes; it's closer to hardware but human-readable.
- Efficient programming requires minimizing memory usage and optimizing the use of AC and temporary registers.

Final Simplified Example: $Z := X + Y$

HDL

```
AC := M(X)      // Load X
DR := AC        // Save X into DR
AC := M(Y)      // Load Y
AC := AC + DR   // Add X (from DR) to Y (in AC)
M(Z) := AC      // Store result in Z
```

Assembly

```
LD X          // Load X into AC
MOV DR, AC    // Save AC to DR
LD Y          // Load Y into AC
ADD           // Add DR to AC
ST Z          // Store result in Z
```